Transformers Overview

What are Transformers?

Transformers are a type of deep learning model architecture introduced in the paper "Attention is All You Need" by Vaswani et al. in 2017. Transformers have become the foundation for many state-of-the-art models in natural language processing (NLP) and computer vision due to their ability to handle sequential data more efficiently than previous models like RNNs (Recurrent Neural Networks) and LSTMs (Long Short-Term Memory networks).

Key Components of Transformers:

1. **Self-Attention Mechanism:**
   - Self-attention allows the model to weigh the importance of different words in a sentence when encoding a specific word. This mechanism helps the model capture long-range dependencies and relationships between words, regardless of their distance in the input sequence.
2. **Multi-Head Attention:**
   - Multi-head attention extends the self-attention mechanism by allowing the model to focus on different parts of the sequence from multiple perspectives simultaneously. This increases the model's capacity to learn diverse representations.
3. **Positional Encoding:**
   - Since Transformers don't inherently understand the order of input tokens (like words), positional encodings are added to the input embeddings to provide information about the position of each token in the sequence.
4. **Feedforward Neural Networks:**
   - After attention mechanisms, the output is passed through a feedforward neural network to further process the information.
5. **Encoder-Decoder Architecture:**
   - The original Transformer model consists of an encoder and a decoder. The encoder processes the input sequence, while the decoder generates the output sequence, often used for tasks like machine translation.

Why Use Transformers?

1. **Parallelization:**
    - Unlike RNNs, which process sequences step-by-step, Transformers can process all tokens in a sequence simultaneously, enabling faster training through parallelization. This is a significant advantage when working with large datasets.
2. **Handling Long-Range Dependencies:**
    - Transformers excel at capturing long-range dependencies in data due to their self-attention mechanism. This is especially important in NLP tasks where the meaning of a word can depend on distant words in a sentence.
3. **Scalability:**
    - Transformers scale well with data and computational resources, making them suitable for large-scale models like GPT (Generative Pre-trained Transformer) and BERT (Bidirectional Encoder Representations from Transformers), which have millions or even billions of parameters.
4. **State-of-the-Art Performance:**
    - Transformers have set new benchmarks in various NLP tasks, including text classification, sentiment analysis, machine translation, and more. Their ability to learn complex patterns and representations has led to significant advancements in these areas.
5. **Versatility:**
    - Although originally designed for NLP, Transformers have been adapted for various tasks, including image processing (Vision Transformers), speech recognition, and even reinforcement learning. Their flexibility allows them to be applied to a wide range of problems.
6. **Pre-training and Fine-tuning:**
    - The architecture of Transformers allows for effective pre-training on large datasets and fine-tuning on specific tasks, leading to models that generalize well across different tasks with minimal additional training.

Use Cases of Transformers:

- **Language Models:** GPT, BERT, T5, and other language models use transformers

for tasks like text generation, summarization, and question answering.

- **Machine Translation:** Models like MarianMT and T5 use transformers to translate text between languages.
- **Text Classification:** BERT and its variants are used for sentiment analysis, topic classification, and other text classification tasks.
- **Vision Transformers (ViT):** Transformers have been adapted for image classification, object detection, and other computer vision tasks.
- **Speech Recognition:** Transformers are used in models like Speech-to-Text for converting spoken language into text.

In summary, Transformers are powerful and versatile models that have revolutionized deep learning, particularly in NLP and computer vision. Their ability to process sequences in parallel, handle long-range dependencies, and scale effectively makes them the architecture of choice for many state-of-the-art models today.

Lecture Notes

Architecture of a Transformer model

1_F3ze0JiQNPsLTN8tDFKfaQ.png

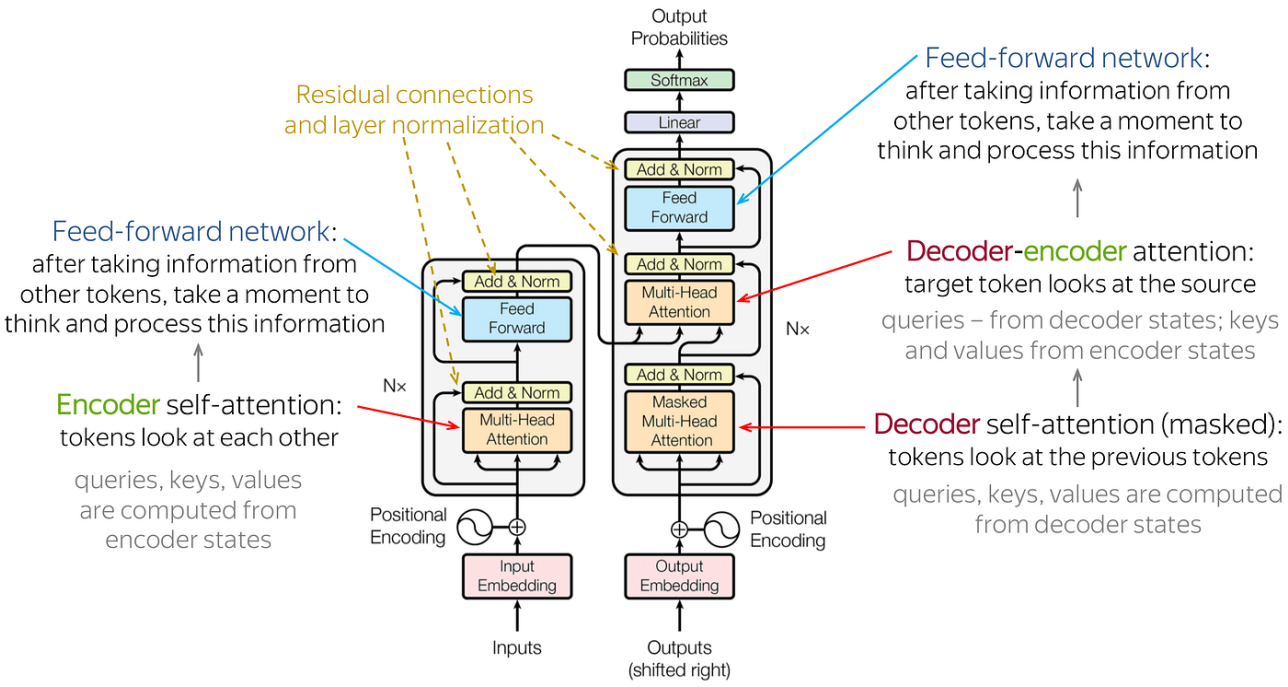


Image Source

Example:

Let's say we're translating the sentence "The cat sits on the mat" from English to French

Let's say we're translating the sentence "The cat sits on the mat" from English to French. The Transformer model helps achieve this by going through several steps.

1. **Input Embedding**:

- **What it is:** Before the model processes the sentence, each word in "The cat sits on the mat" is converted into a numerical form called an embedding. This embedding represents the meaning of each word in a way that the model can understand.
- **Example:** "The" might be converted to a vector like [0.5, 0.1, ...], "cat" to [0.3, 0.4, ...], and so on.

2. **Positional Encoding**:

- **What it is:** Transformers don't have a natural understanding of the order of words, so positional encoding is added to embeddings to help the model know the position of each word in the sentence.
- **Example:** The word "The" at the start gets a positional encoding, helping the model recognize that it's the first word in the sequence.

3. **Encoder (Left Side) – Processing the Input Sentence**:

- **Encoder Self-Attention:**
  - **What it does:** Each word looks at all other words in the sentence to understand its context better.
  - **Example:** The word "cat" might focus more on the words "sits" and "mat" to understand that it's an animal that is sitting on something.
- **Feed-Forward Network:**
  - **What it does:** After the self-attention step, the model processes the combined information from all words through a small neural network to refine its understanding.
  - **Example:** The word "cat" is now better understood in context, and this refined information is passed forward.
- **Residual Connections and Layer Normalization:**
  - **What they do:** These help ensure that the model retains the original information and stabilizes learning, so it doesn't forget anything crucial as it processes the sentence.
- **Nx (repeated layers):**
  - **What it does:** The self-attention and feed-forward network steps are repeated multiple times (e.g., 6 layers). This allows the model to build a

repeated multiple times (e.g., 6 layers). This allows the model to build a deeper understanding of the sentence.
    - **Example:** After several layers, the model deeply understands that "The cat sits on the mat" involves an action by a cat on a mat.

4. **Decoder (Right Side) – Generating the Output Sentence**:

- **Decoder Self-Attention:**
    - **What it does:** While generating each word in the French sentence, the decoder looks at the words it has already generated to make sure they make sense together.
    - **Example:** When generating the French translation "Le chat," the model makes sure "chat" fits well after "Le."
- **Decoder-Encoder Attention:**
    - **What it does:** The decoder looks back at the encoded English sentence to ensure the French words it's generating correctly match the meaning of the original English words.
    - **Example:** While generating the word "chat," the model ensures it matches the English word "cat" from the original sentence.
- **Feed-Forward Network:**
    - **What it does:** Similar to the encoder, this step refines the information after each attention process.
    - **Example:** The word "chat" is refined and prepared for output.
- **Nx (repeated layers):**
    - **What it does:** Just like in the encoder, these layers are repeated to improve the quality of the output translation.

5. **Final Output**:

- **Softmax & Linear Layer:**
    - **What it does:** After processing through all decoder layers, the final word is chosen from the possible vocabulary, using probabilities.
    - **Example:** The model might output "chat" as the translation of "cat," with high probability.

The final French translation "Le chat est assis sur le tapis" (The cat sits on the mat) is generated word by word, with the Transformer model ensuring that each word makes sense in context and correctly reflects the meaning of the original English sentence.

Summary:

- **Encoder** processes the entire input sentence by focusing on each word in the context of others.
- **Decoder** generates the output sentence by looking at both the original input and the words it has already generated.
- **Self-Attention** helps the model focus on the right words, while **Feed-Forward Networks** refine the information.
- **Positional Encoding** ensures the model understands the order of words.

This architecture is powerful because it handles sequences efficiently, captures long-range dependencies, and processes multiple words simultaneously, leading to high-quality translations and other NLP tasks.

Self-Attention

The **Self-Attention** mechanism is a critical component of the Transformer architecture, and it's what allows the model to understand the relationships between different words in a sentence, regardless of their position. Let's break down how the Self-Attention layer works in detail.

What is Self-Attention?

Self-Attention allows a model to weigh the importance of different words in a sentence relative to each other. Essentially, it helps the model determine which words are relevant when processing a particular word. This is especially useful for capturing long-range dependencies in sequences.

How Does It Work?

Imagine we have a simple sentence: "The cat sat on the mat." We'll see how Self-Attention processes this sentence to understand the relationships between words.

Step 1: Embedding the Words

- First, each word in the sentence is converted into a vector (an embedding) that captures its meaning in a numerical form.
- For example, "The" might be represented as `[0.5, 0.2, 0.1]`, "cat" as `[0.3, 0.8, 0.6]`, and so on.

Step 2: Creating Query, Key, and Value Vectors

- For each word, we generate three different vectors: a **Query (Q)** vector, a **Key (K)** vector, and a **Value (V)** vector.
    - **Query** represents the word we're focusing on (e.g., "cat").
    - **Key** represents other words that we want to compare with (e.g., "sat," "mat").
    - **Value** represents the information to be passed on if the word is found relevant.

  These vectors are computed through linear transformations (multiplying the word embeddings by weight matrices):

$$[Q = W_Q \cdot X, \quad K = W_K \cdot X, \quad V = W_V \cdot X]$$

  Where:

    - (W_Q), (W_K), and (W_V) are learned weight matrices.
    - (X) is the word embedding.

  For simplicity, let's assume:

    - "The" has $(Q = [1, 0])$, $(K = [0.6, 0.4])$, and $(V = [0.8, 0.2])$.
    - "cat" has $(Q = [0.5, 1])$, $(K = [0.2, 0.8])$, and $(V = [0.4, 0.6])$.
    - And so on for the other words.

Step 3: Calculating Attention Scores

- The attention score between two words is calculated by taking the **dot product** of their Query and Key vectors. This measures how much focus one word should have on another.

  For example, to see how much "cat" should focus on "sat":

$$[\text{Attention Score} = Q_{\text{cat}} \cdot K_{\text{sat}}]$$

  The dot product gives a scalar value that indicates the relevance. The higher the score, the more attention one word pays to another.

Step 4: Applying Softmax to Get Attention Weights

- The attention scores for a word across all other words are passed through a **softmax function** to normalize them into probabilities (weights) that sum to 1.

  If "cat" has attention scores with "The," "sat," and "mat" as `[2.5, 1.0, 0.5]`, the softmax would turn this into something like `[0.7, 0.2, 0.1]`. This means "cat" should pay 70% attention to "The," 20% to "sat," and 10% to "mat."

Step 5: Calculating the Weighted Sum of Values

Step 5: Calculating the Weighted Sum of Values

- Each word's Value vector is then weighted by these attention scores. The weighted sum of these values gives the final output for that word.

  For "cat":

  $$[\text{Output Vector} = 0.7 \cdot V_{\text{The}} + 0.2 \cdot V_{\text{sat}} + 0.1 \cdot V_{\text{mat}}]$$

  This output vector is what the model uses as the new representation of the word "cat," considering the context of the sentence.

Step 6: Multi-Head Self-Attention (Optional)

- In practice, instead of just one set of Q, K, V vectors, multiple sets (called "heads") are used. Each head captures different aspects of the relationships between words. These heads are combined at the end to produce the final output.

Why is Self-Attention Important?

1. **Capturing Context:** Self-Attention allows each word to focus on the relevant words in the sentence, regardless of their position. For example, in "The cat sat on the mat," the word "sat" might focus more on "cat" and "mat" to understand the context.
2. **Handling Long Sentences:** Unlike RNNs, which can struggle with long sequences, Self-Attention can directly relate any two words in the sentence, making it easier to capture long-range dependencies.
3. **Parallelization:** Since Self-Attention doesn't rely on the sequential nature of sentences like RNNs, it can process words in parallel, speeding up training and inference.

Summary of Self-Attention:

- **Input:** Word embeddings.
- **Process:** Generate Q, K, V vectors, calculate attention scores, apply softmax, and compute the weighted sum of values.
- **Output:** A new representation of the input sentence that captures the relationships between words.

Self-Attention is powerful because it helps the Transformer model understand the importance of each word relative to others, leading to better performance in tasks like translation, text generation, and more.

Sure! Let's break down the concepts of Query (Q), Key (K), and Value (V) vectors using a

simple analogy.

Analogy: Finding Books in a Library

Imagine you're in a library, and you're looking for books that are most relevant to your current interest, which is about "history."

- **Query (Q):** The Query is like the question you have in mind when searching for a book. In this case, your query is "history." The Query vector represents what you're looking for.
- **Key (K):** The Key is like the label on each book in the library that tells you what it's about. For example, one book might have the key "history," another might have "science," and another might have "fiction." The Key vector represents the topic or the main idea of each word in a sentence.
- **Value (V):** The Value is the actual content inside each book. It's the information that you're interested in reading if the book matches what you're looking for. The Value vector represents the detailed information that the model uses to make sense of the sentence.

How They Work Together:

1. **Matching Query with Keys:** When you go to the library, you use your query "history" to look at the labels (keys) of the books. If the label (key) matches your query, you decide that this book is relevant.
2. **Retrieving Values:** Once you've found the relevant books based on the keys, you open them up and read the content (value) inside. If the label says "history" and your query was "history," you'll read that book more carefully.

In a Transformer Model:

- The **Query vector (Q)** represents the word you're focusing on. For example, if you're processing the word "cat" in the sentence, its Query vector tells the model what "cat" is looking for in the other words.
- The **Key vector (K)** is created for every word in the sentence. It helps the model figure out which words in the sentence might be relevant to the Query.
- The **Value vector (V)** holds the actual information about each word that the model uses to understand the context.

Example in a Sentence:

Let's take the sentence, "The cat sat on the mat."

- If you're processing the word "cat":
  - **Query (Q):** "What other words should 'cat' pay attention to?"
  - **Keys (K):** "Is this word relevant to 'cat'?" For each word ("The," "sat," "on," etc.), the Key vector answers this.
  - **Values (V):** "What does each word mean?" These vectors contain the information that will be weighted based on how relevant the Key is to the Query.

The model uses these vectors to decide which words "cat" should focus on to understand the sentence's meaning better. This helps the model create a richer and more accurate representation of the sentence.

Blog

Multi Head Attention

Self-Attention Mechanism

The **Self-Attention** mechanism is a crucial component in transformer models, allowing them to weigh the importance of different words in a sentence when encoding a word, considering its context. Here's how it works step by step:

1. **Input Representation**: Each word in a sentence is first embedded into a vector. Let's say you have a sentence with `n` words, each represented by a vector of dimension `d_model`.
2. **Query, Key, and Value Vectors**: For each word (represented by a vector), three different vectors are derived through linear transformations:
   - **Query (Q)**: Represents the word you are trying to understand in context.
   - **Key (K)**: Represents potential words that could match with the query.
   - **Value (V)**: Contains the actual information/content of the words.

   Mathematically, these are calculated as:
   $$[Q = XW_Q, \quad K = XW_K, \quad V = XW_V]$$
   where $(X)$is the input word embedding and (W_Q), (W_K) and $(W_V)$are learnable weight matrices.
3. **Attention Scores**: The attention score for each word is computed by taking the dot product of the query vector with all key vectors, including itself, to determine how

much focus each word should have on others in the sequence.

$$[\text{Attention Score} = \frac{Q \cdot K^T}{\sqrt{d_k}}]$$

where (d_k) is the dimension of the key vectors. The score is scaled by $(\sqrt{d_k})$ to maintain stable gradients.

4. **Softmax**: The attention scores are passed through a softmax function to convert them into a probability distribution. This determines how much attention each word should pay to every other word.
5. **Weighted Sum**: Each word's representation (value vector) is weighted by the attention scores. The output for each word is a weighted sum of all the value vectors, where the weights are the attention probabilities.

Multi-Head Self-Attention

The **Multi-Head Self-Attention** mechanism enhances the model's ability to focus on different aspects of the sentence simultaneously by having multiple attention heads. Here's how it works:

1. **Multiple Heads**: Instead of computing a single set of Q, K, and V vectors, the model computes several sets (e.g., 8 or 16). Each set is called a "head" and captures different relationships or features from the input.
2. **Parallel Attention Mechanisms**: Each head performs the self-attention process independently. This allows the model to focus on different parts of the sequence and learn various relationships.
3. **Concatenation and Projection**: The outputs from all the heads are concatenated and then passed through another linear transformation (using a matrix (W_O) to combine the information from each head into a single vector for each word.

   Mathematically:

   $$[\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \text{head}_2, \ldots, \text{head}_h)W_O]$$

   where $(\text{head}_i = \text{Attention}(QW_Q^i, KW_K^i, VW_V^i))$.

Benefits of Multi-Head Self-Attention

- **Different Aspects of Focus**: Each head can attend to different parts of the sentence, allowing the model to learn more complex patterns.
- **Better Representation**: By combining the outputs of different heads, the model gets a richer representation of the input sequence, leading to better performance in tasks like translation, summarization, and more.

Summary

Self-Attention with Multi-Head allows transformers to dynamically weigh the importance of each word in a sequence in relation to every other word. The multi-head mechanism further enriches this process by enabling the model to attend to multiple aspects of the input simultaneously, capturing complex patterns in data.

Feed Forward Neural Network (FFNN)

A **Feed Forward Neural Network (FFNN) with Multi-Head Attention** is a key component of transformer models, where the multi-head attention mechanism is followed by a feed-forward neural network. This combination helps the model process information in a way that is both context-aware (thanks to attention) and non-linear (thanks to the feed-forward layers).

Overview of the Process

1. **Multi-Head Self-Attention**:
    - **Purpose**: To allow the model to focus on different parts of the input sequence and capture various relationships between the words.
    - **Mechanism**:
        - The input sequence is passed through multiple attention heads.
        - Each attention head computes a different attention score, focusing on different aspects of the sequence.
        - The outputs from all heads are concatenated and linearly transformed to produce a single output for each word in the sequence.
2. **Add & Normalize**:
    - The output from the multi-head attention is added to the original input (using a residual connection), and the result is normalized (Layer Normalization). This helps stabilize the training process and allows the model to learn more effectively.
3. **Feed Forward Neural Network (FFNN)**:
    - **Purpose**: To apply a non-linear transformation to the attention outputs, enabling the model to capture complex patterns in the data.
    - **Structure**:
        - The output from the attention mechanism (after normalization) is

passed through a feed-forward neural network.

- This FFNN typically consists of two linear transformations with a ReLU activation function in between:

$$[\text{FFNN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2]$$

where (W_1), (W_2) are weight matrices, and (b_1), (b_2) are biases.

- The first layer increases the dimensionality (hidden layer) and the second layer reduces it back to the original size.
- For example, if the input dimension is (d_{model}), the hidden layer might have a size of $(4 \times d_{\text{model}})$, and the output would return to (d_{model}).

4. **Add & Normalize (again)**:
    - After the FFNN, another residual connection is applied, adding the FFNN's output to the input of the FFNN. This is followed by layer normalization.

Detailed Steps in the Process

1. **Input Embedding**: The sequence of words is embedded into vectors of dimension (d_{model}).
2. **Multi-Head Attention**:
    - For each word in the sequence, create multiple sets of query, key, and value vectors.
    - Compute the attention scores and apply softmax to get the weighted sum of value vectors.
    - Concatenate the outputs of all heads and apply a linear transformation.
3. **Residual Connection & Layer Normalization**:
    - Add the output of the multi-head attention back to the original input, then normalize the result.
4. **Feed Forward Neural Network**:
    - Pass the normalized output through the feed-forward neural network.
    - Apply a ReLU activation after the first linear transformation, then apply a second linear transformation to return to the original dimension.
5. **Residual Connection & Layer Normalization**:
    - Add the output of the FFNN back to the input of the FFNN, then normalize the result.

Significance of Combining FFNN with Multi-Head Attention

- **Multi-Head Attention**: This allows the model to focus on different parts of the sequence, capturing various dependencies and relationships between words.
- **Feed Forward Network**: The FFNN applies non-linear transformations, enabling the model to learn complex patterns that simple linear transformations could not capture.
- **Residual Connections**: These help mitigate issues like vanishing gradients and enable the model to learn deeper representations without the risk of performance degradation.
- **Layer Normalization**: Helps stabilize and accelerate training by ensuring that the outputs of each layer are normalized, improving gradient flow and convergence.

Summary

In transformer models, the combination of **Multi-Head Self-Attention** and **Feed Forward Neural Networks** allows for both context-aware processing (thanks to attention) and complex pattern learning (thanks to non-linear transformations). This architecture is the backbone of models like BERT, GPT, and others, enabling them to perform well on a wide range of NLP tasks.

transformer_multi-headed_self-attention-recap.png

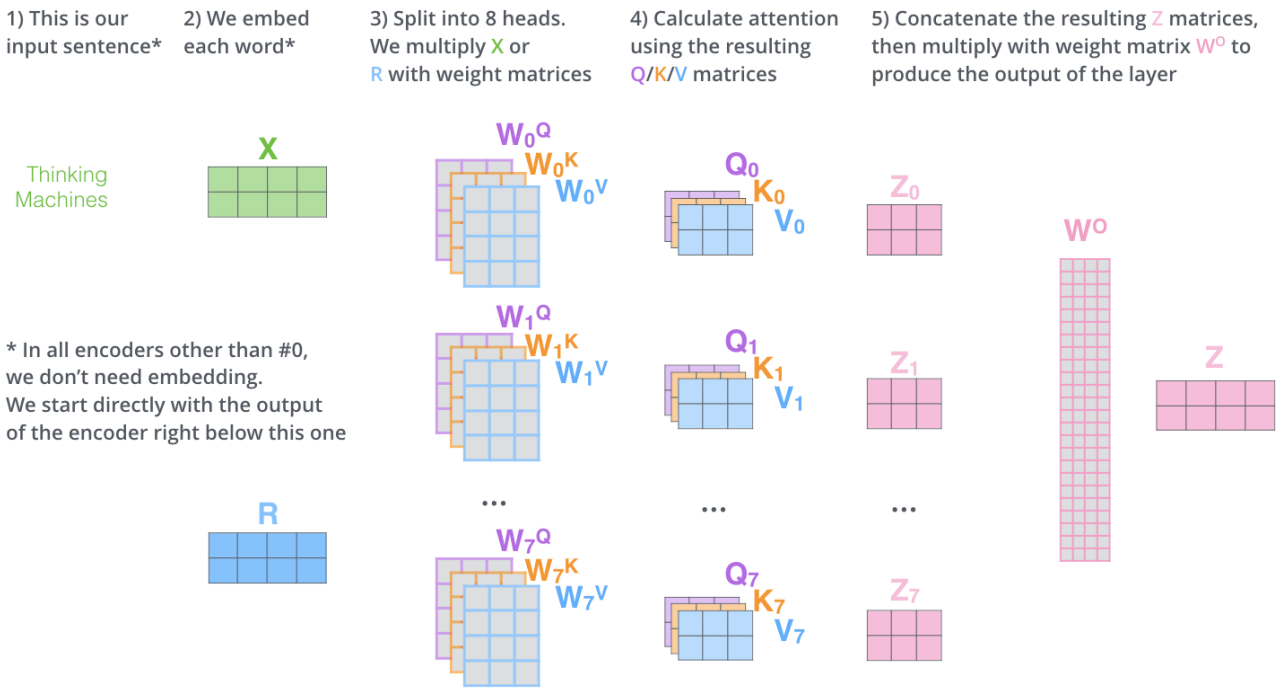


Image Source

Positional Encoding

transformer_positional_encoding_vectors.png

Image Source

What is Positional Encoding?

Positional Encoding is a technique used in transformer models to inject information about the position of words in a sequence into the model. Unlike Recurrent Neural Networks (RNNs) or Convolutional Neural Networks (CNNs), transformers do not have any inherent notion of sequence order because they process input data in parallel. Positional encoding provides the necessary information to the model about the relative or absolute position of each word in the sequence.

Why Positional Encoding is Necessary

In language processing tasks, the order of words in a sentence is crucial for understanding the meaning. For example, the sentences "The cat chased the dog" and "The dog chased the cat" contain the same words but have entirely different meanings due to the order of the words. Without a way to represent this order, a transformer model wouldn't be able to correctly interpret the sentence.

How Positional Encoding Works

Positional encoding adds positional information to the input embeddings. This is done by generating a unique positional vector for each position in the sequence, which is then added to the word embeddings. The positional encoding vectors are designed so that each position in the sequence is represented uniquely, and the model can learn the order of the words.

Mathematical Representation

For a sequence of length (n) and embedding dimension (d_{model}), the positional encoding (PE) is a vector of the same dimension as the word embedding. The positional encoding for each position (pos) in the sequence is calculated using the following formulas:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}}\right)$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}}\right)$$

(pos) is the position of the word in the

- (pos) is the position of the word in the sequence.
- (i) is the dimension index.
- (d_{model}) is the dimension of the embedding vector.

The use of sine and cosine functions with different wavelengths allows the model to easily learn to attend to relative positions between words.

Key Properties

1. **Unique Positional Representation**: Each position in the sequence has a unique encoding.
2. **Smooth Changes**: Small changes in position result in small changes in the encoding, which allows the model to generalize well to unseen positions or longer sequences.
3. **Relative Position Information**: The use of sine and cosine functions ensures that the positional encodings carry information about the relative positions of words.

Where Positional Encoding is Used in Transformers

In transformer models, positional encoding is added to the input embeddings before they are fed into the encoder and decoder layers.

1. **Input Embeddings**: Each word in the input sequence is first converted into a fixed-size vector through embedding. This embedding contains the semantic meaning of the word but no information about its position in the sequence.
2. **Adding Positional Encoding**: The positional encoding is added element-wise to the word embeddings:
   $$\text{Input to the Transformer} = \text{Word Embedding} + \text{Positional Encoding}$$
   This combined vector is then passed into the transformer model.
3. **Encoder and Decoder**: Both the encoder and decoder use the positional encodings to process the input data. Since the encoder processes the entire input sequence simultaneously, the positional encodings help it understand the order of the words.

Example: Understanding Sentence Structure

Consider the sentence: "The quick brown fox jumps over the lazy dog."

- The model first converts each word into an embedding vector (e.g., using a pre-trained embedding like Word2Vec or GloVe).
- Positional encodings are generated for each word's position in the sentence (1st word, 2nd word, etc.).

word, 2nd word, etc.).

- These encodings are added to the word embeddings.
- The combined vectors (word embedding + positional encoding) are then passed through the transformer model.

This process allows the transformer to understand that "The quick brown fox" should be processed as the subject of the sentence and that "jumps over the lazy dog" is the action and the object.

Advantages of Positional Encoding

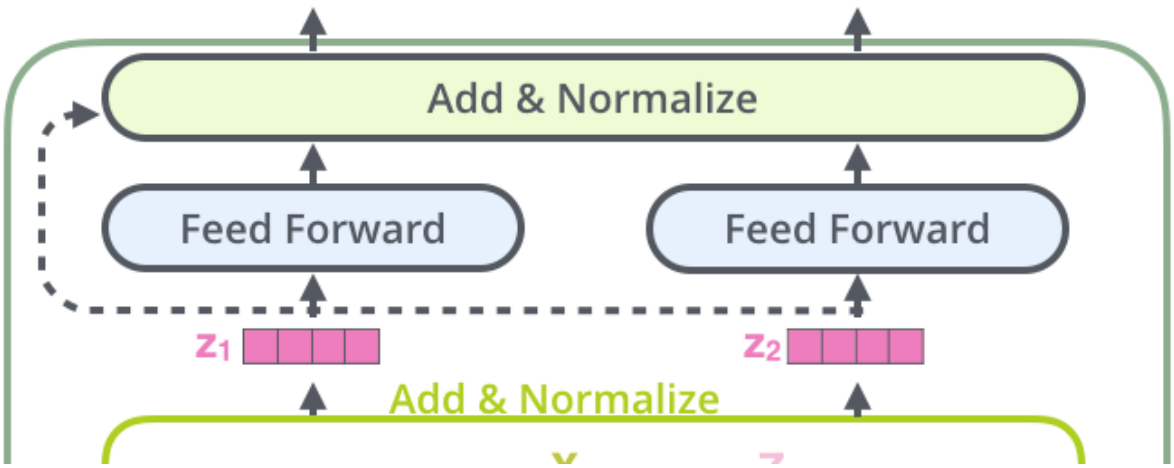
1. **Parallelization**: Positional encoding allows the transformer to process sequences in parallel, unlike RNNs, which process sequences sequentially.
2. **Flexibility**: The use of sine and cosine functions makes it easier for the model to generalize to sequences of varying lengths, even those longer than the sequences seen during training.
3. **Efficiency**: Positional encoding is computationally efficient and integrates seamlessly with the transformer architecture.

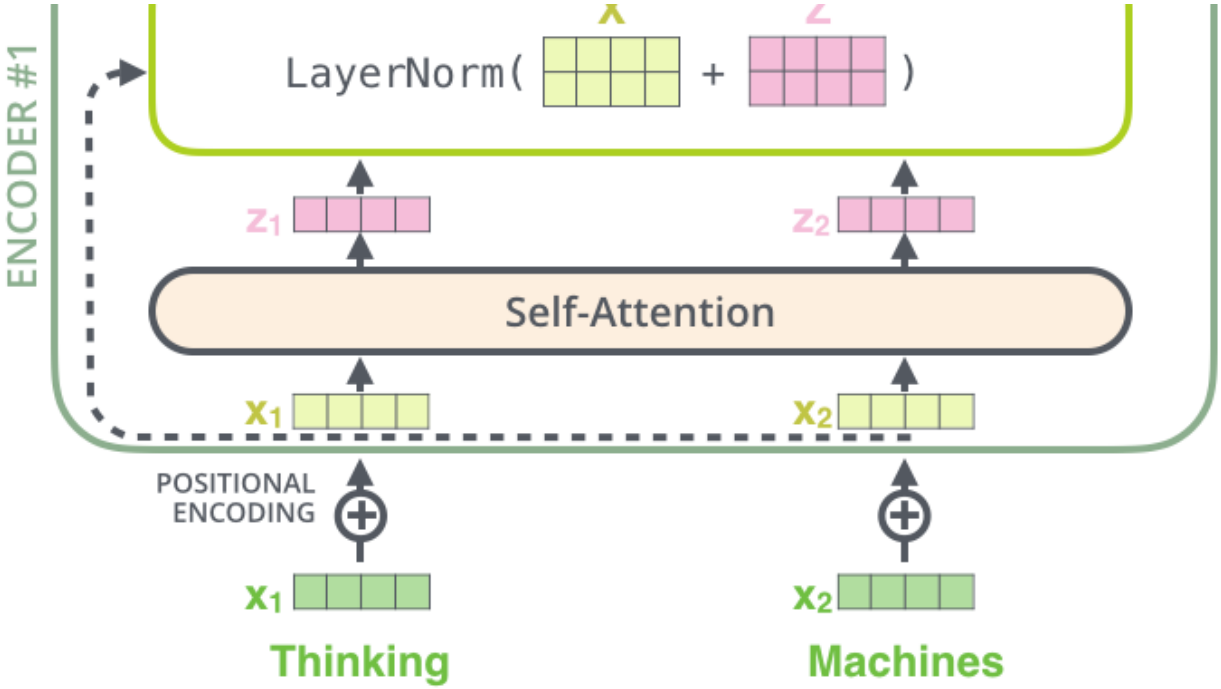
Summary

Positional Encoding is an essential component of transformer models that provides information about the order of words in a sequence, compensating for the transformer's lack of inherent sequence awareness. It is added to word embeddings before the data is processed by the transformer, enabling the model to understand the structure and meaning of sentences.

Layer Normalization and Residual Connections

transformer_resideual_layer_norm_2.png





Overview of Transformers and Previous Topics

- **Self-Attention**: A mechanism that allows the model to focus on different parts of the input sequence, converting fixed vectors from the input into contextual vectors. It helps in understanding the relationships between different words in a sequence.
- **Multi-Head Attention**: An extension of self-attention that allows the model to jointly attend to information from different representation subspaces. It uses multiple attention heads to enhance the model's capability.
- **Positional Encoding**: Since transformers do not have a built-in notion of sequence order, positional encoding is added to the input embeddings to provide information about the position of each word in the sequence.
- **Contextual Vectors**: Vectors obtained after applying the self-attention mechanism, which incorporate contextual information from the entire sequence.

Layer Normalization and Residual Connections in Transformers

Introduction to Layer Normalization

- **Layer Normalization**: A normalization technique applied within the transformer architecture to stabilize and speed up training. It normalizes the inputs across the features rather than across the batch, which differs from batch normalization.
- **Purpose**: Helps maintain the stability of the network during training, especially with

- **Purpose**: Helps maintain the stability of the network during training, especially with deep architectures like transformers.
- **Application**: Layer normalization is applied after adding the input embeddings with positional encoding and before feeding the data to the multi-head attention mechanism.

Add and Normalize: Integrating Residual Connections with Layer Normalization

transformer-architecture-encoder-decoder.png

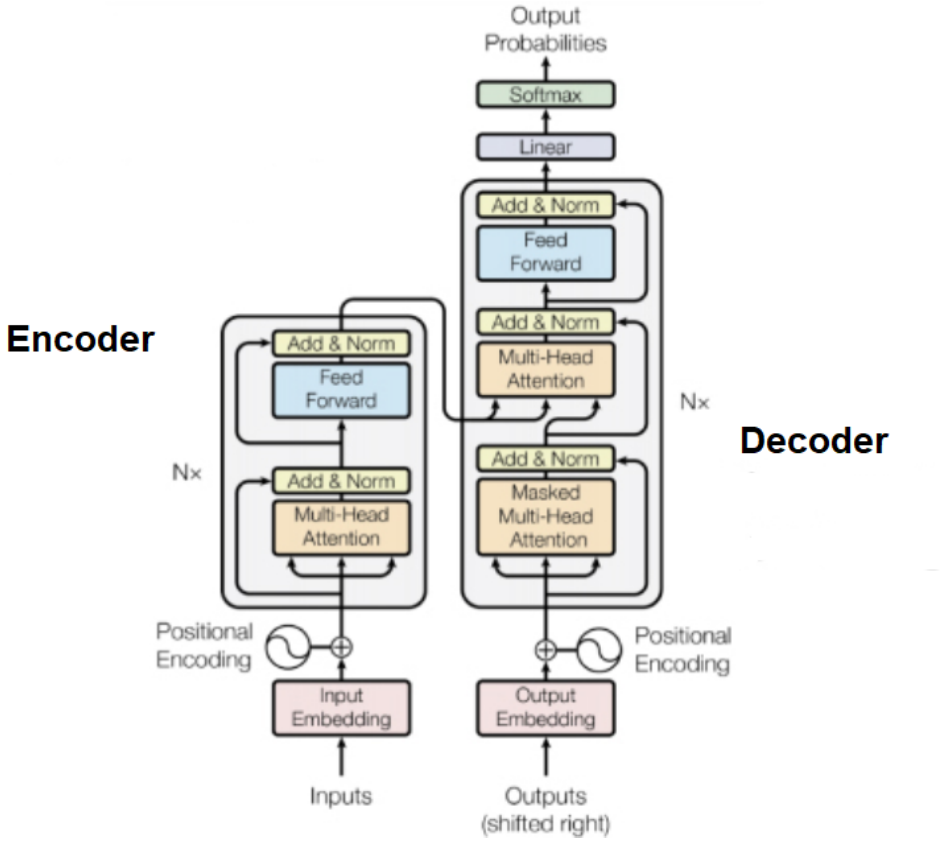


Image Source

Residual Connections

- **Residual Connections**: A technique that adds the input of a layer to the output of the layer before passing it to the next layer. It helps in preserving the original input information and provides additional signals to the subsequent layers.
- **Why Residual Connections?**: They prevent the vanishing gradient problem and make it easier to train deep networks by ensuring that gradients flow directly through the network. In transformers, this is particularly important due to the depth and complexity of the architecture.

Add and Normalize Process

- **Add Operation**: The output from the multi-head attention or the feed-forward network is added to the input embeddings (or the output from the previous layer). This addition is what is referred to as the residual connection.
- **Normalization**: After the addition, layer normalization is applied to ensure that the resulting vectors maintain a stable mean and variance, which aids in the model's convergence during training.
- **Importance**: The combination of residual connections and layer normalization allows transformers to maintain a balance between preserving input information and ensuring the output is stable and normalized.

Detailed Explanation of Layer Normalization

What is Normalization?

- **Normalization in Deep Learning**: A technique used to scale inputs or outputs so that they have a mean of zero and a standard deviation of one. This helps in improving the stability and performance of the model during training.
- **Batch Normalization vs. Layer Normalization**:
  - **Batch Normalization**: Normalizes across the batch dimension and is commonly used in traditional neural networks.
  - **Layer Normalization**: Normalizes across the features dimension for each input individually, which is more suitable for models like transformers where batch sizes can vary.

Mathematical Intuition

- **Standard Scaling (Z-score)**: Involves subtracting the mean and dividing by the standard deviation for each feature, ensuring that the data is centered around zero with a standard deviation of one.
- **Formula**:
$$[z = \frac{x - \mu}{\sigma}]$$
where (x) is the input, (μ) is the mean, and (σ) is the standard deviation.
- **Impact on Training**:
  - **Improved Training Stability**: Normalized data helps in stabilizing the training process by preventing the vanishing or exploding gradient problems.

- **Faster Convergence**: Since the data is zero-centered and has a uniform scale, the model can converge faster during training.
- **Stable Updates in Backpropagation**: Ensures that during backpropagation, the updates to the weights are stable, leading to more consistent training.

Example: Normalization in Practice

Neural Network Example

- **Inputs and Hidden Layers**: Consider a neural network with two input features (e.g., house size and number of rooms) and a hidden layer with two neurons.
- **Pre-Normalization**: Before feeding the inputs to the network, normalization is applied to ensure that the input features are on a similar scale.
- **Data Example**:
  - House Size: 1200 sq ft, 1500 sq ft, 2000 sq ft
  - Number of Rooms: 2, 3, 3.5
  - Price: 45 lakhs, 70 lakhs, 80 lakhs
- **Normalization Impact**:
  - After normalization, the features are transformed such that their mean is 0, and the standard deviation is 1.
  - This transformation helps in ensuring that the network can process the inputs more effectively, leading to better performance and faster convergence.

Layer Normalization, (γ) (gamma) and (β) (beta) are learnable parameters that are used to scale and shift the normalized output, respectively. Here's a detailed explanation:

Role of (γ) (Scale) and (β) (Shift) in Layer Normalization

- **Layer Normalization**: As a recap, layer normalization normalizes the inputs across the features for each data point individually. After normalizing, each input has a mean of zero and a standard deviation of one.
- **Mathematical Formula**: The normalized output $(\hat{x})$ is calculated as:

$$[\hat{x} = \frac{x - \mu}{\sigma}]$$

  where:
  - (x) is the input to the layer.
  - (μ) is the mean of the inputs.

- (σ) is the standard deviation of the inputs.

- **Inclusion of (γ) and (β)**: After normalization, the output is further transformed by scaling it with $(\gamma)$and shifting it with (β):
$$[y = \gamma \cdot \hat{x} + \beta]$$
where:
  - (γ) (gamma) is the **scale** parameter.
  - (β) (beta) is the **shift** parameter.

Purpose of (γ) and (β)

- **Learnable Parameters**: Both (γ) and (β) are learnable parameters, meaning they are adjusted during the training process to optimize the model's performance. They allow the model to scale and shift the normalized output, which provides the flexibility to recover the original input distribution if needed.
- **Flexibility in Representation**:
  - **Without (γ) and (β)**: The output would always have a mean of 0 and a standard deviation of 1 after normalization, which might be restrictive for the model's ability to represent complex patterns.
  - **With (γ) and (β)**: The model can learn to adjust the normalized output to any desired scale and shift, providing the necessary flexibility to learn a wide range of representations.
- (γ) **(Scale)**: Controls the scaling of the normalized output, allowing the model to learn the appropriate amplitude for the features.
- (β) **(Shift)**: Controls the shifting of the normalized output, allowing the model to adjust the mean value of the features.

These parameters are crucial for enhancing the expressiveness and flexibility of the transformer model, enabling it to perform better across different tasks.

Let's break down the concept of (γ) (scale) and (β) (shift) in Layer Normalization with a simple example.

Example: Layer Normalization with and without (γ) and (β)

Suppose we have a small neural network layer with three features (neurons) in the output: (x_1), (x_2), and (x_3). The output of this layer for a single input data point is:

$$[x = [3.0, 5.0, 7.0]]$$

Step 1: Calculate the Mean ((μ)) and Standard Deviation ((σ))

Step 1: Calculate the Mean (μ) and Standard Deviation (σ)

First, we compute the mean (μ) and standard deviation (σ) of the input:

$$\mu = \frac{3.0 + 5.0 + 7.0}{3} = \frac{15.0}{3} = 5.0$$

$$\sigma = \sqrt{\frac{(3.0 - 5.0)^2 + (5.0 - 5.0)^2 + (7.0 - 5.0)^2}{3}} = \sqrt{\frac{4.0 + 0.0 + 4.0}{3}} = \sqrt{\frac{8.0}{3}}$$

Step 2: Normalize the Inputs

Next, we normalize the inputs:

$$\hat{x}_1 = \frac{3.0 - 5.0}{1.63} \approx -1.23$$

$$\hat{x}_2 = \frac{5.0 - 5.0}{1.63} = 0.0$$

$$\hat{x}_3 = \frac{7.0 - 5.0}{1.63} \approx 1.23$$

So, the normalized output (\hat{x}) is:

$$\hat{x} = [-1.23, 0.0, 1.23]$$

Step 3: Apply (γ) (Scale) and (β) (Shift)

Without scaling and shifting, the normalized values always have a mean of 0 and a standard deviation of 1. However, we might want the model to adjust these values during training. This is where (γ) (scale) and (β) (shift) come in.

Let's assume:

- $(\gamma = [2.0, 0.5, 1.0])$ (scaling factors for each feature)
- $(\beta = [1.0, 2.0, 0.5])$ (shifting factors for each feature)

Now, we apply these to the normalized values:

$$y_1 = \gamma_1 \cdot \hat{x}_1 + \beta_1 = 2.0 \cdot (-1.23) + 1.0 \approx -2.46 + 1.0 = -1.46$$

$$y_2 = \gamma_2 \cdot \hat{x}_2 + \beta_2 = 0.5 \cdot 0.0 + 2.0 = 0.0 + 2.0 = 2.0$$

$$y_3 = \gamma_3 \cdot \hat{x}_3 + \beta_3 = 1.0 \cdot 1.23 + 0.5 \approx 1.23 + 0.5 = 1.73$$

So, the final output after applying (\gamma) and (\beta) is:

$$y = [-1.46, 2.0, 1.73]$$

Summary of the Example:

- **Without (γ) and (β):** The normalized output $(\hat{x})$ is constrained to have a mean of

- **Without (\gamma) and (\beta)**: The normalized output $(\hat{x})$ is constrained to have a mean of 0 and a standard deviation of 1.

$$[\hat{x} = [-1.23, 0.0, 1.23]]$$

- **With (\gamma) and (\beta)**: The model can learn to adjust the normalized output to fit the data better, as seen in the final output:

$$[y = [-1.46, 2.0, 1.73]]$$

By learning the appropriate (γ) and (β) during training, the model gains more flexibility in how it represents data, leading to potentially better performance.

Here are the notes based on the transcript you provided:

Layer Normalization Example in Transformer Layers

Overview

- In this example, we'll walk through the process of Layer Normalization applied to a single token in sequence within a transformer layer.
- The example focuses on understanding how Layer Normalization works, particularly how the (γ) (scale) and (β) (shift) parameters influence the normalized output.

Token Embeddings

- Consider a token representing the word "cat," which is embedded into a vector: $([2.4, 4.4, 6.0, 8.0])$.
- The parameters (γ) (scale) and (β) (shift) are initialized. (γ) is the learned scale parameter, and (β) is the shift parameter.

Step-by-Step Calculation

1. **Compute the Mean (μ)**
   - Formula: $(\mu = \frac{1}{N} \sum_{i=1}^{N} x_i)$
   - For the vector $([2.4, 4.4, 6.0, 8.0])$, the mean is calculated as: $$[\mu = \frac{2.4 + 4.4 + 6.0 + 8.0}{4} = \frac{20.8}{4} = 5.2]$$
2. **Compute the Variance ((σ^2))**
   - Formula: $(\sigma^2 = \frac{1}{N} \sum_{i=1}^{N} (x_i - \mu)^2)$
   - Calculation for each element:

$$[\sigma^2 = \frac{1}{4} \left[(2.4 - 5.2)^2 + (4.4 - 5.2)^2 + (6.0 - 5.2)^2 + (8.0 - 5.2)^2 \right]]$$

$$[= \frac{1}{4}[7.84 + 0.64 + 0.64 + 7.84] = \frac{16.96}{4} = 4.24]$$

3. **Normalize the Inputs**
   - Formula: $(\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}})$
   - (ϵ) is a small value (e.g., $(1e^{-5})$) to avoid division by zero.
   - Standard deviation $(\sigma = \sqrt{4.24} \approx 2.06)$.
   - Normalize each element:

$$[\hat{x}_1 = \frac{2.4 - 5.2}{2.06} \approx -1.36]$$
$$[\hat{x}_2 = \frac{4.4 - 5.2}{2.06} \approx -0.39]$$
$$[\hat{x}_3 = \frac{6.0 - 5.2}{2.06} \approx 0.39]$$
$$[\hat{x}_4 = \frac{8.0 - 5.2}{2.06} \approx 1.36]$$

4. **Apply Scale and Shift (Gamma and Beta)**
   - Formula: $(y_i = \gamma \hat{x}_i + \beta)$
   - Given that $(\gamma = [1.0, 1.0, 1.0, 1.0])$ and $(\beta = [0.0, 0.0, 0.0, 0.0])$, the final output remains the same as the normalized values: $$[y = [-1.36, -0.39, 0.39, 1.36]]$$
   - If (γ) and (β) values were different, they would adjust the normalized output accordingly.

Integration in Transformer Architecture

- **Positional Encoding:** Added to token embeddings before the normalization step.
- **Multi-head Attention:** Applies Layer Normalization before and after the attention mechanism.
- **Feed Forward Neural Network:** After normalization, the output is passed to a feed-forward neural network.
- **Add and Normalize:** The output of the feed-forward neural network is added back to the original signal and normalized again.
- **Decoder:** The normalized output is sent to the decoder, which will be discussed in future topics.

Key Points

- Layer Normalization ensures that the input to the next layer has a mean of 0 and a variance of 1.
- The (γ) (scale) and (β) (shift) parameters allow the model to learn optimal

- The (γ) (scale) and (β) (shift) parameters allow the model to learn optimal transformations for the normalized output.
- Layer Normalization is essential for stabilizing the training process and improving model performance.

Double-click (or enter) to edit

Complete Encoder Transformer Architecture

We'll delve into the full encoder architecture of the Transformer model, building on the foundational components we've previously discussed, such as self-attention, multi-head attention, positional encoding, and layer normalization. Let's break down the entire encoder process step by step, as outlined in the original research paper.

Overview of the Encoder Architecture

The Transformer architecture involves multiple encoders, typically six, as described in the paper. Each encoder is composed of several key components that work together to process the input sequence and generate meaningful representations.

Step-by-Step Breakdown:

1. **Input Sequence and Embedding**:
    - The process begins with the input sequence, which is a series of tokens (e.g., words).
    - Each token is converted into a fixed-size vector using text embedding techniques. According to the research paper, the dimension of these embedding vectors is 512.
    - Positional encoding is added to these embeddings to retain the order of the sequence, as the Transformer does not inherently consider the sequence order.
2. **Self-Attention Mechanism**:
    - After embedding, the input is passed through the self-attention layer.
    - Self-attention allows the model to weigh the importance of different tokens in the sequence relative to each other, helping the model focus on relevant parts of the input.
3. **Multi-Head Attention**:

- Multi-head attention involves multiple self-attention mechanisms running in parallel (eight heads in this case).
- Each head learns different aspects of the relationships between tokens, enhancing the model's ability to capture complex dependencies.

4. **Residual Connections and Layer Normalization**:
   - The output of the multi-head attention is then combined with the original input (embedding + positional encoding) through a residual connection. This helps in maintaining the original signal while adding the learned features from the self-attention.
   - Layer normalization is applied to stabilize and speed up the training process, ensuring that the input distribution to the next layer is normalized.
5. **Feed-Forward Neural Network (FFNN)**:
   - The normalized output is then passed through a feed-forward neural network.
   - The FFNN typically has one hidden layer with 512 units and applies a non-linear transformation, enhancing the model's ability to learn complex representations.
   - The output of the FFNN is again passed through a residual connection and normalized before being sent to the next encoder.
6. **Stacking Encoders**:
   - The output from one encoder is passed as input to the next encoder. This stacking allows the model to learn progressively more abstract features at each layer.
   - By the time the input passes through all six encoders, the model has built a rich and deep representation of the sequence.

Key Concepts Explained:

- **Residual Connections**:
  - Residual connections, or skip connections, are crucial for addressing the vanishing gradient problem, which can occur in deep networks like this one with multiple layers.
  - They allow gradients to flow directly through the network during backpropagation, ensuring that the gradients remain sufficiently large and improving the overall convergence rate.
  - This mechanism enables the training of deeper networks by facilitating smoother training and avoiding issues like vanishing or exploding gradients.

- **Importance of FFNN**:
  - The feed-forward neural network serves to add non-linearity and complexity to the model's representations, which helps in learning more intricate patterns in the data.
  - Even though the model is already performing attention operations, the FFNN provides additional capacity for the model to learn and transform the input features before they are passed on to subsequent layers.

Conclusion:

The encoder architecture of the Transformer is designed to effectively handle complex sequence-to-sequence tasks, such as language translation, by stacking multiple layers of encoders. Each component, from self-attention to feed-forward networks, plays a critical role in building a deep, expressive model capable of learning intricate relationships in the input data.

Overview of the Decoder in Transformers

- **Purpose**: The decoder is responsible for generating the output sequence one token at a time using the encoder output and previously generated tokens.
- **Key Components**: The decoder consists of three primary components:
  1. **Masked Multi-Head Self-Attention**: This component handles attention by focusing on parts of the input while masking certain parts to ensure the correct generation sequence.
  2. **Multi-Head Attention (Encoder-Decoder Attention)**: This is where the decoder attends to the encoder's output, allowing it to focus on relevant parts of the input sequence.
  3. **Feed Forward Neural Network**: Standard component responsible for processing the attention results and contributing to the final output.

Encoder-Decoder Interaction

- **Input to Encoder**: Inputs (e.g., words or tokens) are passed through the encoder, which processes them and produces an output.
- **Output Generation**: The decoder takes this output and generates tokens one by one, forming the final sequence.

Detailed Decoder Working

Detailed Decoder Working

- **Step-by-Step Mechanism**:
    1. **Input Embedding**: Converts the input tokens into vectors.
    2. **Positional Embedding**: Adds information about the position of tokens in the sequence.
    3. **Linear Projection**: Projects inputs into queries (Q), keys (K), and values (V) needed for attention calculations.
    4. **Scaled Dot-Product Attention**: Calculates attention scores to determine the importance of different tokens.
    5. **Mask Application**: In the Masked Multi-Head Self-Attention, masks are applied to prevent future tokens from being seen during training.
    6. **Multi-Head Attention Mechanism**: Combines multiple attention heads for a richer representation.
    7. **Concatenation and Final Linear Projection**: Merges the results and projects them into the final dimensions.
    8. **Residual Connection and Layer Normalization**: Helps stabilize training by adding the input back to the output and normalizing it.

Key Differences Between Encoder and Decoder

- **Simultaneous Processing vs. Sequential Generation**: While the encoder processes all tokens simultaneously, the decoder generates them sequentially, one at a time.

Aspect	Encoder
Purpose	Processes the input sequence to generate hidden representations.
Structure	Consists of multiple identical layers, each with two sub-layers: 1. Self-Attention Mechanism 2. Feed-Forward Neural Network
Attention Mechanisms	Uses self-attention to focus on different parts of the input sequence.
Input and Output	Takes in the entire input sequence and outputs a set of vectors representing the encoded inp
Processing Sequence	Processes the entire sequence in parallel.
Training vs. Inference	Functions similarly during both training and inference, processing the input sequence in one

Masked Multi-Head Self-Attention

Blog

---

1. Overview

Masked Multi-Head Self-Attention is a specialized mechanism used primarily in the **decoder** of Transformer models, such as those used in Natural Language Processing (NLP) tasks. This mechanism allows the model to generate sequences (like text) by attending to different parts of the input while maintaining a causal relationship, ensuring that predictions for a particular position do not depend on future tokens.

Masked Multi-Head Self-Attention is a mechanism used in the **decoder** part of Transformer models, especially in tasks like text generation. It allows the model to focus on different parts of the input (like words in a sentence) when predicting the next word, but with a crucial twist: it ensures that the model only looks at words it has already seen and not the ones that come after.

2. Where It Is Used

- **Decoder in Transformers**: Masked Multi-Head Self-Attention is used within the decoder part of a Transformer architecture. It plays a crucial role in autoregressive models like GPT, where the task is to generate text or sequences one token at a time.
- **Language Modeling**: In tasks like language modeling, where the model predicts the next word in a sequence, masked attention ensures that the prediction for the next word doesn't "cheat" by looking at future words.
- **Prevents "Cheating"**: When generating a sentence, the model shouldn’t look ahead to see the future words. Masked attention ensures that when predicting a word, the model can only consider the words before it and not the ones after.
- **Handles Multiple Perspectives**: The "multi-head" part means the model looks at the input from different angles (or "heads") to capture various relationships between the words. Each head focuses on different aspects of the input, like which words are most important to the current word being generated.

3. How It Is Used

- **Preventing Information Leakage**: The core idea behind masked attention is to prevent the model from attending to future tokens that it should not have access to when predicting the next token in a sequence.
- **Autoregressive Generation**: During sequence generation, the decoder predicts each word one at a time. Masking is applied to ensure that each word is predicted only based on the previous words and not the words that come after it.

4. Full Working of Masked Multi-Head Self-Attention

4. Full Working of Masked Multi-Head Self-Attention

4.1. Self-Attention Mechanism Recap

Self-attention allows the model to weigh the importance of different words in a sequence when processing each word. It computes attention scores based on the query, key, and value matrices derived from the input.

4.2. Masking in Self-Attention

- **Mask Application**: In masked self-attention, a mask is applied to the attention scores before the softmax operation. This mask is usually a triangular matrix (lower triangular) that prevents the model from attending to future positions in the sequence.
- **Attention Score Calculation**: For each word, the attention scores are computed with all other words, but the mask ensures that future words have an attention score of negative infinity (`-inf`) before applying the softmax function. This effectively sets the attention to zero for future tokens.

4.3. Steps in Masked Multi-Head Self-Attention

1. **Input Embedding**: The input tokens are first converted into embedding vectors.
2. **Positional Encoding**: Since the Transformer model doesn't inherently understand the order of tokens, positional encodings are added to the embeddings.
3. **Linear Projections (Q, K, V)**: The embeddings are linearly projected to generate the query (Q), key (K), and value (V) matrices.
4. **Scaled Dot-Product Attention**: The attention scores are calculated using the formula:

$$[\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + \text{mask}\right)V]$$

   where `mask` is applied to ensure that future tokens are not attended to.
5. **Mask Application**: The mask is applied to the attention scores to prevent the model from considering future tokens.
6. **Multi-Head Attention**: The process is repeated across multiple heads to capture different relationships in the sequence, and the outputs are concatenated and linearly projected to form the final attention output.
7. **Residual Connection and Layer Normalization**: The output from the multi-head attention is added back to the input (residual connection) and then normalized.

5. Types of Masking

- **Look-Ahead Mask (Causal Mask)**: This is the most common type of masking used in masked self-attention. It prevents the model from attending to future tokens by setting attention scores to `-inf` for tokens that should not be seen.

  Example: For a sequence of length 4, the mask might look like this:

$$\begin{bmatrix} 0 & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

- **Padding Mask**: Used to ignore padding tokens in the input sequence. Padding tokens are used to make all sequences in a batch the same length, but they do not contain useful information.

  Example: If a sequence is `[word1, word2, word3, PAD]`, the padding mask will ensure that the model does not attend to the PAD token.

6. Flow of Working in Masked Multi-Head Self-Attention

1. **Input Tokens**: A sequence of input tokens is fed into the decoder.
2. **Embedding & Positional Encoding**: Tokens are converted into embeddings, and positional information is added.
3. **Query, Key, Value Calculation**: Linear transformations are applied to generate Q, K, and V matrices.
4. **Mask Application**: A mask is applied to the attention scores to prevent the decoder from attending to future tokens.
5. **Attention Calculation**: The attention scores are calculated, and the masked scores are passed through softmax to obtain the attention weights.
6. **Weighted Sum**: The attention weights are used to compute a weighted sum of the value vectors, resulting in the output for each head.
7. **Concatenation & Projection**: The outputs of all heads are concatenated and linearly projected to form the final output.
8. **Residual & Normalization**: A residual connection adds the input to the output, followed by layer normalization.
9. **Final Output**: The final output is passed to the next layer or the next step in the sequence generation process.

7. Importance of Masked Multi-Head Self-Attention

- **Sequential Generation**: It ensures that the model generates sequences in a causal, left-to-right manner, which is crucial for tasks like text generation and

causal, left-to-right manner, which is crucial for tasks like text generation and translation.

- **Preventing Information Leakage**: By masking future tokens, it prevents the model from using information that it should not have access to, maintaining the integrity of the generation process.

Simple Example

Imagine you're writing a story, and you can only see the words you've already written, not the words you're about to write. Masked Multi-Head Self-Attention works similarly: it helps the model decide what the next word should be by looking at the words it already knows, ensuring it doesn't peek at the future ones.

In summary, Masked Multi-Head Self-Attention is a fundamental component in Transformer-based decoders, enabling them to generate sequences in a controlled, autoregressive manner by carefully attending to past and current tokens while ignoring future ones.

Encoder-Decoder Attention

What is the Encoder-Decoder Attention?

In the Transformer model, the Encoder and Decoder are two key components. The **Encoder** processes the input sequence (like a sentence in one language), while the **Decoder** generates the output sequence (like the translated sentence in another language).

Encoder-Decoder Attention is a mechanism in the Decoder that allows it to focus on different parts of the input sequence (processed by the Encoder) when generating each word of the output sequence.

How does it work?

1. **Input Sequence**: The input sentence is first converted into a set of vectors by the Encoder. Each word or token in the sentence is represented as a vector (a list of numbers that captures the meaning of the word in context).
2. **Attention Mechanism**:
    - The attention mechanism helps the Decoder focus on different parts of the input sentence while generating each word of the output sentence.
    - Instead of treating all words in the input equally, the Decoder can "pay more

- Instead of treating all words in the input equally, the Decoder can "pay more attention" to the words that are more relevant for producing the next word in the output.

3. **Multi-Head Attention**:
   - The attention mechanism used is called **Multi-Head Attention**. It uses multiple "heads," or parallel attention layers, to capture different types of relationships between the input and output tokens.
   - Each attention head looks at the input sequence from a different perspective, capturing various types of dependencies (like focusing on different words or combinations of words).
4. **Combining the Information**:
   - The outputs of all the attention heads are combined and passed through the remaining layers of the Decoder to generate the next word in the output sequence.
   - This process is repeated for each word in the output sentence.

Why is it important?

Encoder-Decoder Attention is important because it allows the model to use information from the entire input sequence when generating each word of the output. This is crucial for tasks like translation, where understanding the context provided by the entire sentence can be key to producing accurate and fluent translations.

In summary, **Encoder-Decoder Attention** is a way for the Decoder to focus on the most relevant parts of the input sequence when generating each word in the output sequence, making the Transformer model more effective for tasks that involve understanding and generating sequences of text.

Full flow from the Encoder to the Decoder

Let's walk through the full flow from the Encoder to the Decoder in a Transformer model, focusing on how information is transferred and utilized by the **Encoder-Decoder Attention** mechanism.

Step 1: Input Sequence Processing by the Encoder

1. **Input Embedding**:
   - The input sequence (like a sentence) is first passed through an embedding layer that converts each word or token into a vector (a list of numbers

representing the word in a multi-dimensional space).

2. **Positional Encoding**:
    - Since the Transformer does not inherently understand the order of words (it doesn't have the sequential nature of RNNs), **Positional Encoding** is added to the embeddings. This helps the model understand the position of each word in the sequence.
3. **Multi-Head Self-Attention** (within the Encoder):
    - The self-attention mechanism in the Encoder allows each word in the sequence to focus on other words in the same sequence, capturing relationships between them.
    - Multiple attention heads are used to capture different types of relationships between the words.
4. **Feed-Forward Network**:
    - After attention, each word's representation is passed through a feed-forward neural network to further process the information.
    - This step is followed by **Add & Norm** layers, which help stabilize and enhance the training process.
5. **Output of the Encoder**:
    - The final output of the Encoder is a set of vectors, where each vector represents a word in the input sequence but with enriched information, thanks to the attention mechanisms and the feed-forward network.

Step 2: Passing Encoder's Output to the Decoder

- The output from the Encoder is now ready to be fed into the Decoder. These outputs serve as the "memory" or context that the Decoder will use to generate the output sequence.

Step 3: Decoder Processing with Encoder-Decoder Attention

1. **Input Embedding (in the Decoder)**:
    - The target sequence (the sequence the model is trying to generate, such as a sentence in another language) is also converted into embeddings, just like the input sequence was.
2. **Positional Encoding**:
    - Similar to the Encoder, positional encodings are added to the target

sequence embeddings to help the model understand the order of the words.

3. **Masked Multi-Head Self-Attention** (within the Decoder):
    - The Decoder applies self-attention to the target sequence. However, it is masked so that the model can't "cheat" by looking at future tokens in the sequence while predicting the current token. This ensures that the model generates the output sequence one token at a time, in order.
4. **Encoder-Decoder Attention**:
    - **This is the critical step where the Decoder focuses on the output from the Encoder**.
    - The Decoder now has the encoded representations (context vectors) from the Encoder. In the **Encoder-Decoder Attention** layer, the Decoder attends to these vectors while generating each word in the target sequence.
    - This means that for each word it tries to generate, the Decoder can look back at the entire input sequence and decide which parts are most relevant.
    - The attention mechanism here operates similarly to the self-attention mechanism in the Encoder, but instead of focusing on the target sequence, it focuses on the encoded input sequence.
5. **Feed-Forward Network**:
    - After the attention layers, the Decoder also passes the attended representations through a feed-forward network, similar to the Encoder.
6. **Output Generation**:
    - Finally, the processed information goes through a linear layer and a softmax function to predict the next word in the output sequence.
    - This process repeats for each word in the target sequence until the entire sequence is generated.

Summary of the Flow:

- **Encoder** processes the input sequence into context-rich vectors.
- These vectors are then passed to the **Decoder**, where:
    - The **Decoder** first processes its own input sequence with self-attention.
    - It then uses **Encoder-Decoder Attention** to attend to the Encoder's output, determining which parts of the input sequence to focus on when generating each word.
- This flow ensures that the model generates an output sequence that is contextually

aligned with the input sequence, making it effective for tasks like translation, summarization, and more.

Final Layers of Transformers: Linear and Softmax

The final **Linear** and **Softmax** layers in the Decoder of the Transformer model play a crucial role in generating the actual output sequence (e.g., translated text). Here's how they work:

Context: Where Are We in the Decoder?

After the Decoder has processed the target sequence through its layers—masked multi-head self-attention, Encoder-Decoder attention, and feed-forward networks—we end up with a set of vectors. Each vector corresponds to a position in the output sequence being generated (e.g., each word or token).

Step 1: Linear Layer

- **Purpose**: The purpose of the Linear layer is to convert the processed vectors from the Decoder into a format that can be used to predict the next word in the output sequence.
- **Operation**:
    - The output from the Decoder at each position is a vector of a certain dimension (let's say 512, for example).
    - The vocabulary size of the language model (e.g., English, with 30,000 possible words/tokens) determines the number of possible words the model can predict.
    - The Linear layer is essentially a fully connected layer that transforms the Decoder's output vector (of dimension 512) into a new vector of dimension equal to the vocabulary size (e.g., 30,000).
- **Example**:
    - If the vocabulary size is 30,000, the Linear layer will convert each vector from the Decoder into a vector with 30,000 elements. Each element of this vector corresponds to a score (or logit) for a particular word in the vocabulary.

Step 2: Softmax Layer

- **Purpose**: The Softmax layer converts the logits (scores) generated by the Linear

- **Purpose**: The Softmax layer converts the logits (scores) generated by the Linear layer into probabilities. These probabilities indicate the likelihood of each word in the vocabulary being the next word in the output sequence.
- **Operation**:
  - The Softmax function takes the vector of logits (e.g., the 30,000 scores) and normalizes them into a probability distribution.
  - This means that all the scores are converted into values between 0 and 1, and the sum of these probabilities is 1.
  - The word with the highest probability is typically chosen as the next word in the output sequence.
- **Mathematical Explanation**:
  - If the output of the Linear layer for a certain position is a vector $([z_1, z_2, \ldots, z_{30000}])$, the Softmax function will compute the probability of each word (i) as: $$[P_i = \frac{e^{z_i}}{\sum_{j=1}^{30000} e^{z_j}}]$$
  - Here, (e^{z_i}) represents the exponentiation of the logit for word (i), and the denominator ensures that all probabilities sum to 1.

Step 3: Output Generation

- Once the probabilities are computed by the Softmax layer, the word with the highest probability is selected as the next word in the output sequence.
- This word is then fed back into the Decoder (during training) or used to generate the next word in the sequence (during inference or generation).

Summary:

- The **Linear layer** converts the Decoder's output vectors into logits, with each logit corresponding to a word in the vocabulary.
- The **Softmax layer** then converts these logits into probabilities, indicating how likely each word in the vocabulary is to be the next word in the sequence.
- The word with the highest probability is chosen as the next word in the output sequence, completing one step in the sequence generation process.

Flow

Linear Layer

- **Concept of the Linear Layer**
  - Definition: A fully connected neural network layer that projects decoder

vectors to a larger vector space (logits).

- **Mathematical Representation**
  - Explanation of how the Linear layer transforms decoder outputs into logits.
  - Formula: $(\text{Logits} = W \cdot \text{Decoder\_Output} + b)$
- **Vocabulary Size and Logits**
  - How the size of the logits vector corresponds to the vocabulary size.
  - Example: If vocabulary size is 10,000, logits vector has 10,000 elements.

Softmax Layer

- **Purpose of the Softmax Layer**
  - Converts logits into probabilities.
  - Used for multi-class classification.
- **Mathematical Representation**
  - Formula for Softmax: $(\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}})$
  - Explanation of how it converts logits into a probability distribution.
- **Output Interpretation**
  - Probability distribution over the vocabulary.
  - How the word with the highest probability is selected as the output.

End-to-End flow of a Transformer model

Example:

Task: Translate an English sentence into French.

Input Sentence: "I love cats."

Expected Output Sentence: "J'aime les chats."

Step 1: Input Processing in the Encoder

1. **Tokenization**

- The input sentence "I love cats." is split into tokens: `["I", "love", "cats", "."]`.
- Each token is then converted into indices using a vocabulary: `[14, 65, 200, 5]` (for example)

(for example).

2. **Input Embedding**

- These indices are mapped to embedding vectors of a fixed dimension (e.g., 512).
- Example:
  - "I" → `[0.1, 0.4, ...]` (512-dimensional vector)
  - "love" → `[0.3, 0.7, ...]`
  - "cats" → `[0.6, 0.1, ...]`
  - "." → `[0.2, 0.9, ...]`

3. **Positional Encoding**

- Positional encodings are added to these embeddings to incorporate information about the position of the words within the sentence.

4. **Multi-Head Self-Attention**

- The model applies self-attention to the entire sequence to capture relationships between different words.
- For example, "love" might attend more to "I" than to "cats".

5. **Add & Norm**

- The output of the attention mechanism is added to the original input (a residual connection), and the result is normalized.

6. **Feed-Forward Network**

- Each vector (corresponding to each token) is passed through a fully connected feed-forward network, which is applied independently to each position.
- The output is normalized again.

7. **Output of the Encoder**

- The Encoder produces a sequence of context-aware vectors representing each word from the input sentence.

Step 2: Decoder Processing with Encoder-Decoder Attention

1. **Start with the Target Sequence (Partial Output)**

- The target sequence starts with the first word (or a start token), e.g., "J'".
- This is converted to an embedding, just like the input sentence.

2. **Masked Multi-Head Self-Attention**

- Similar to the Encoder, but with masking applied to prevent the decoder from "seeing" future tokens.

3. **Add & Norm**

- The attention output is added to the input (residual connection) and normalized.

4. **Encoder-Decoder Attention**

- The Decoder then attends to the entire output sequence of the Encoder, deciding which parts of the input sentence to focus on when generating each word.
- For "J'", it might focus on "I"; for "aime," it might focus on "love."

5. **Add & Norm**

- Again, residual connection and normalization are applied.

6. **Feed-Forward Network**

- Each vector is passed through another fully connected feed-forward network.

7. **Final Linear Layer**

- The output from the Decoder is passed through a linear layer to map it to a vector of the same size as the vocabulary.

8. **Softmax**

- The output of the linear layer is passed through a softmax function to convert it into probabilities, predicting the most likely next word.

Step 3: Output Generation (Repeat for Each Word)

- The Decoder continues to generate the next word in the sequence, feeding the previously generated word back into itself.
- This process repeats until the model generates the end token `<end>` or until the sentence is complete.

Example Flow:

1. **Encoder**:
    - Input: "I love cats."
    - Output: Contextualized vectors representing each word in the sentence.
2. **Decoder**:
    - Input: "J'" (start token)

- Output: "J'aime les chats."
- At each step, the Decoder attends to the Encoder's output while generating each subsequent word.

3. **Final Output**:
    - The sequence "J'aime les chats." is generated word by word until the full translation is produced.

The Transformer model uses the attention mechanism extensively in both the Encoder and Decoder to capture dependencies between words, ensuring that the output sentence is both grammatically correct and semantically aligned with the input sentence.